todd_5530_m8

AUTHOR
Reagan Todd

This is a demonstration of how to join tables in SQL and select rows. Created 2026-04-03 by Reagan Todd. The copyright is for educational uses only.

```
R.version.string
```

```
[1] "R version 4.5.2 (2025-10-31)"
```

```
Sys.Date()
```

```
[1] "2026-04-03"
```

```
library(tidyverse)
library(DBI)
library(RSQLite)
library(readr)
```

Use the MIMIC-III Schema.

Q1. Retrieve patient gender and admission time for 10 rows. Hint: Join patients and admissions to get info.

First we need to ensure the database is loaded correctly.

```
con <- dbConnect(SQLite(), "mimic_demo.db")

patients <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/PATIENTS.csv"
)
```

```
Rows: 100 Columns: 8
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (1): gender
dbl  (3): row_id, subject_id, expire_flag
dttm (4): dob, dod, dod_hosp, dod_ssn
```

```
ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
dbWriteTable(con, "patients", patients, overwrite = TRUE)

admissions <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/ADMISSIONS.csv"
)
```

```
Rows: 129 Columns: 19
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (9): admission_type, admission_location, discharge_location, insurance,...
dbl  (5): row_id, subject_id, hadm_id, hospital_expire_flag, has_chartevents...
dttm (5): admittime, dischtime, deathtime, edregtime, edouttime

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
dbWriteTable(con, "admissions", admissions, overwrite = TRUE)
glimpse(patients)
```

```
Rows: 100
Columns: 8
$ row_id      <dbl> 9467, 9472, 9474, 9478, 9479, 9486, 9487, 9489, 9491, 9492…
$ subject_id  <dbl> 10006, 10011, 10013, 10017, 10019, 10026, 10027, 10029, 10…
$ gender      <chr> "F", "F", "F", "F", "M", "F", "F", "M", "M", "F", "M", "F"…
$ dob         <dttm> 2094-03-05, 2090-06-05, 2038-09-03, 2075-09-21, 2114-06-2…
$ dod         <dttm> 2165-08-12, 2126-08-28, 2125-10-07, 2152-09-12, 2163-05-1…
$ dod_hosp    <dttm> 2165-08-12, 2126-08-28, 2125-10-07, NA, 2163-05-15, NA, N…
$ dod_ssn     <dttm> 2165-08-12, NA, 2125-10-07, 2152-09-12, 2163-05-15, 2195-…
$ expire_flag <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
```

```r
glimpse(admissions)
```

```
Rows: 129
Columns: 19
$ row_id               <dbl> 12258, 12263, 12265, 12269, 12270, 12277, 12278, …
$ subject_id           <dbl> 10006, 10011, 10013, 10017, 10019, 10026, 10027, …
$ hadm_id              <dbl> 142345, 105331, 165520, 199207, 177759, 103770, 1…
$ admittime            <dttm> 2164-10-23 21:09:00, 2126-08-14 22:32:00, 2125-1…
$ dischtime            <dttm> 2164-11-01 17:15:00, 2126-08-28 18:59:00, 2125-1…
$ deathtime            <dttm> NA, 2126-08-28 18:59:00, 2125-10-07 15:13:00, NA…
$ admission_type       <chr> "EMERGENCY", "EMERGENCY", "EMERGENCY", "EMERGENCY…
$ admission_location   <chr> "EMERGENCY ROOM ADMIT", "TRANSFER FROM HOSP/EXTRA…
$ discharge_location   <chr> "HOME HEALTH CARE", "DEAD/EXPIRED", "DEAD/EXPIRED…
$ insurance            <chr> "Medicare", "Private", "Medicare", "Medicare", "M…
$ language             <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "POLI", NA, N…
$ religion             <chr> "CATHOLIC", "CATHOLIC", "CATHOLIC", "CATHOLIC", "…
```

```
$ marital_status       <chr> "SEPARATED", "SINGLE", NA, "DIVORCED", "DIVORCED"…
$ ethnicity            <chr> "BLACK/AFRICAN AMERICAN", "UNKNOWN/NOT SPECIFIED"…
$ edregtime            <dttm> 2164-10-23 16:43:00, NA, NA, 2149-05-26 12:08:00…
$ edouttime            <dttm> 2164-10-23 23:00:00, NA, NA, 2149-05-26 19:45:00…
$ diagnosis            <chr> "SEPSIS", "HEPATITIS B", "SEPSIS", "HUMERAL FRACT…
$ hospital_expire_flag <dbl> 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0…
$ has_chartevents_data <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
```

Now we can do a Left join so that we can retain the patient information but add the admission time if available.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
patient_admittime <- dbGetQuery(conn=cox, "
SELECT
  a.subject_id,
  p.gender,
  datetime(a.admittime, 'unixepoch') AS admittime_dt
FROM admissions a
LEFT JOIN patients p
  ON a.subject_id = p.subject_id
LIMIT 10;
")

patient_admittime
```

```
   subject_id gender        admittime_dt
1       10006      F 2164-10-23 21:09:00
2       10011      F 2126-08-14 22:32:00
3       10013      F 2125-10-04 23:36:00
4       10017      F 2149-05-26 17:19:00
5       10019      M 2163-05-14 20:43:00
6       10026      F 2195-05-17 07:39:00
7       10027      F 2190-07-13 07:15:00
8       10029      M 2139-09-22 10:58:00
9       10032      M 2138-04-02 19:52:00
10      10033      F 2132-12-05 02:46:00
```

```
dbDisconnect(cox)
```

Interpretation of the results

We now have a table that shows the subject ID, the patients gender, and the time that the patient was admitted for the first 10 patients.

Interpretation of the code

Same as Module 6, we have to convert the unixepoch date into a format that we as humans can easily identify, that is why we created the admittime_dt to see exactly what date that patient was admitted.

Q2. Get a count of the number of records in the patients table and admissions table. How many rows will be retrieved if you do a cross join for patient and admissions table.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
counts_a <- dbGetQuery(conn=cox, "
SELECT COUNT(*) AS total_admissions FROM admissions;
")

counts_a
```

```
  total_admissions
1              129
```

```
dbDisconnect(cox)
```

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
counts_p <- dbGetQuery(conn=cox, "
SELECT COUNT(*) AS total_patients FROM patients;
")

counts_p
```

```
  total_patients
1            100
```

```
dbDisconnect(cox)
```

Interpretation of output

The admissions data has 129 rows and the patients data has 100 rows. If we were to do a cross-join, we should have 129*100= 12,900 rows. After checking below, we can see that we now have 12,900 rows.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
crossjoin <- dbGetQuery(conn=cox, "
SELECT COUNT(*) AS total_rows
FROM patients p
CROSS JOIN admissions a;
")

crossjoin
```

```
  total_rows
1      12900
```

```r
dbDisconnect(cox)
```

Q3. List all admissions that do not have a diagnosis. Hint: Use LEFT JOIN + WHERE d.icd9_code IS NULL.

Checking that diagnosis database is loaded.

```r
diagnosis <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/DIAGNOSES_ICD.csv"
)
```

```
Rows: 1761 Columns: 5
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): icd9_code
dbl (4): row_id, subject_id, hadm_id, seq_num

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
dbWriteTable(con, "diagnosis", diagnosis, overwrite = TRUE)
glimpse(diagnosis)
```

```
Rows: 1,761
Columns: 5
$ row_id     <dbl> 112344, 112345, 112346, 112347, 112348, 112349, 112350, 112…
$ subject_id <dbl> 10006, 10006, 10006, 10006, 10006, 10006, 10006, 10006, 100…
$ hadm_id    <dbl> 142345, 142345, 142345, 142345, 142345, 142345, 142345, 142…
$ seq_num    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, …
$ icd9_code  <chr> "99591", "99662", "5672", "40391", "42731", "4280", "4241",…
```

Checking the count of null values first

```r
cox <- dbConnect(SQLite(), "mimic_demo.db")
null <- dbGetQuery(conn=cox, "
SELECT COUNT(*) AS missing_icd9
FROM diagnosis
WHERE icd9_code IS NULL;
")

null
```

```
  missing_icd9
1            0
```

```
dbDisconnect(cox)
```

There were no null values, so we should see an empty table here.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
null_icd <- dbGetQuery(conn=cox, "
SELECT a.subject_id, a.hadm_id, d.icd9_code
FROM admissions a
LEFT JOIN diagnosis d
ON a.subject_id = d.subject_id AND a.hadm_id = d.hadm_id
WHERE d.icd9_code IS NULL;
")

null_icd
```

```
[1] subject_id hadm_id    icd9_code
<0 rows> (or 0-length row.names)
```

```
dbDisconnect(cox)
```

Interpretation of output

There were no missing icd_9 codes within this dataset. The first chunk of code checked to see if there were, but then we used a left join to answer the question to show that there are no null icd-9 codes therefore we are left with a table that has 3 columns (subject ID, hadm ID, and icd-9) but 0 rows because there are no null values.